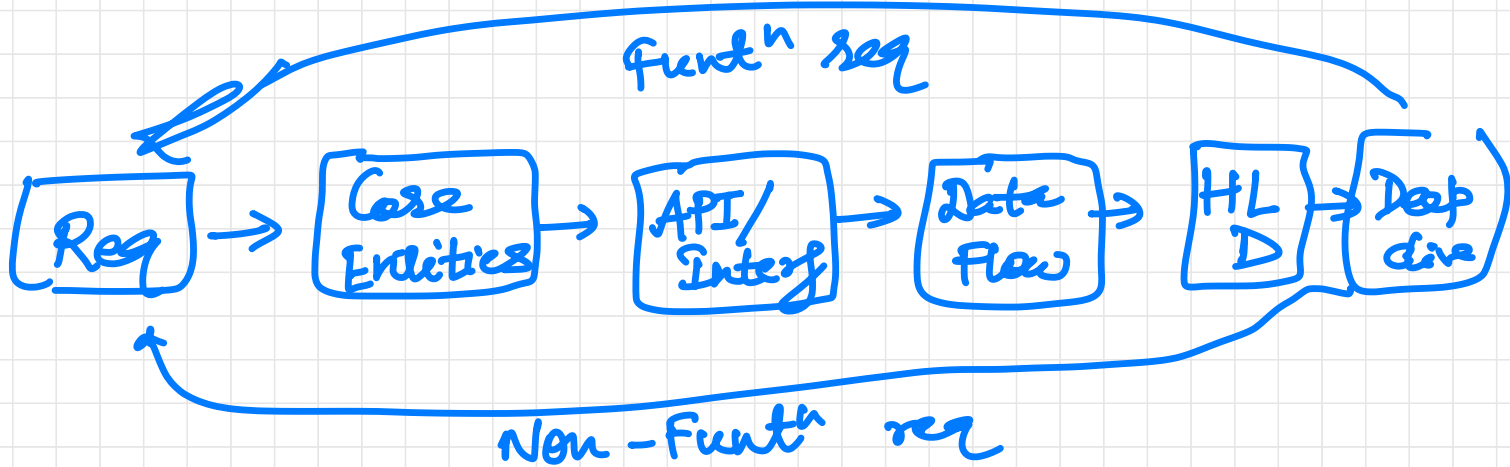



Del. Frame $\begin{cases} \text{key Tech} \\ \text{Pattern} \end{cases}$



Req  **Functi** - Core feature, (3)
Non Functi → Availability, Scalable, Low Latency

① **CAP** (2 of 3 possible)

- Consistent - Read get recent write
- Availability → Non failing
- Partition tolerance → Continue to operate even if w fail

④ Latency

⑤ Durability

⑦ Fault Tolerance

⑧ Compliance

- Banking vs Social w data

② Env. Constr
- mobile/web

③ Scalable

- bursty traffic
- read vs write ratio

⑥ Security

CORE Entities

- actors in sys
- noun / resource
req for Functⁿ
seq

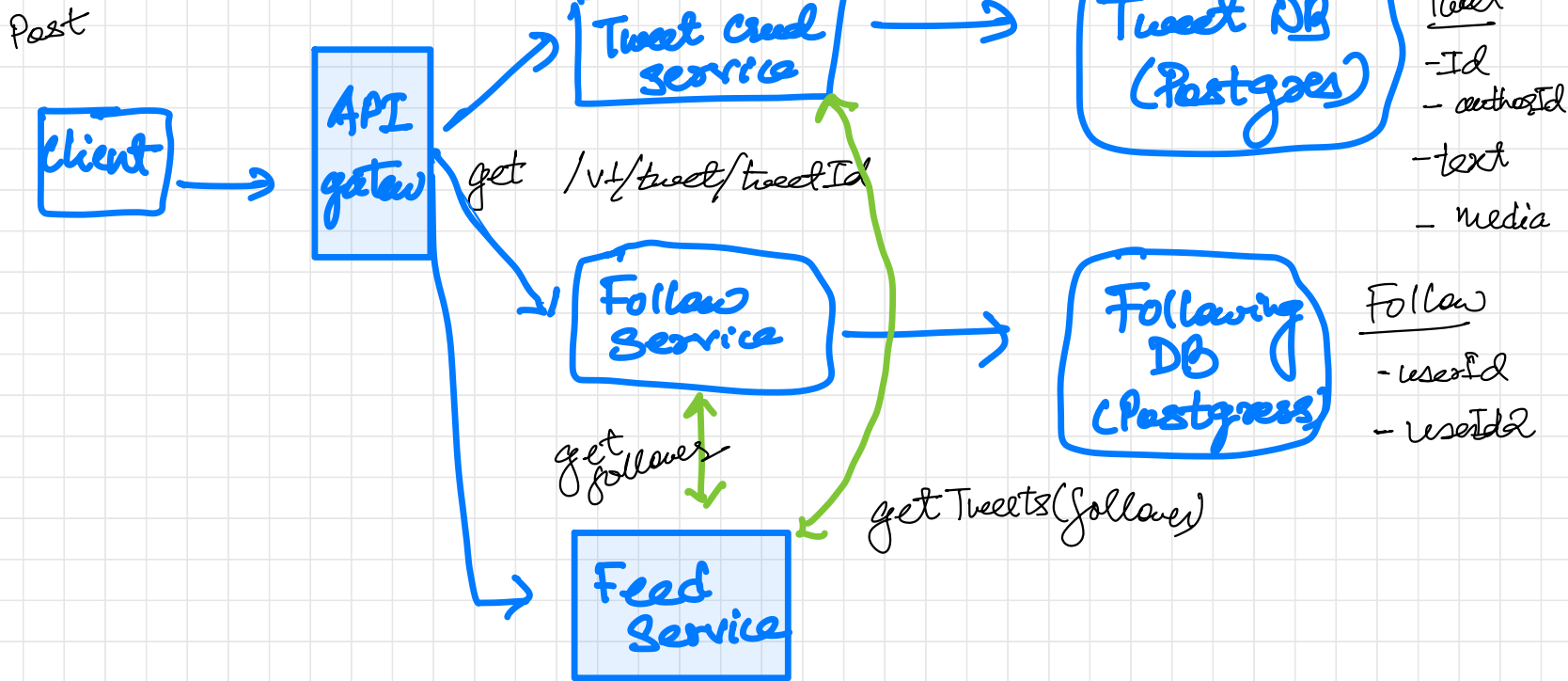
API / Interface

- contract b/w sys
& user ✓
- Restful ✓
- GraphQL
- Wire Protocol

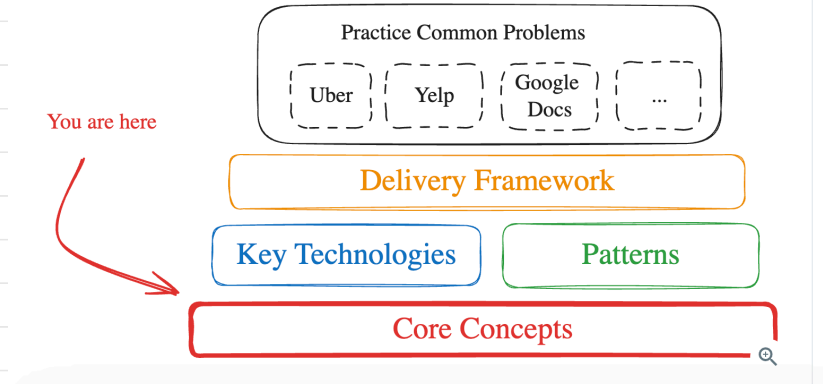
Data Flow

- simple list

HLD

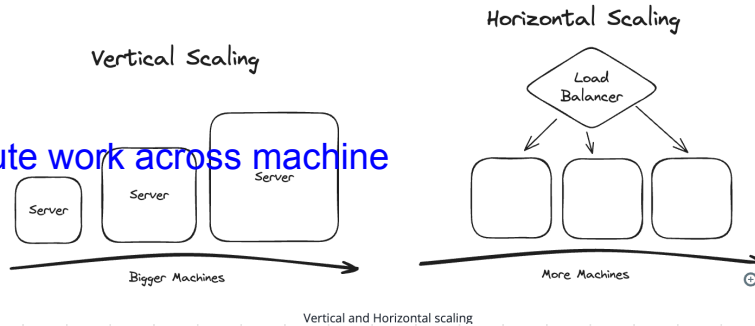


Core



Scaling

1. Horizontal—→ more machine
2. Vertical—→ more resouce to single machine
3. Consistent Hashing—→ distrubute work across machine



Work distribution

- . Horiz Scaling-> Load balancer

- # using hasing map to distribute work across set of nodes

Data Distribution

- # some implies keeping data in memory

- # ways to partition data such single node access data it needs without need to talk another node

- # Anti pattern-> requests fan out to many diff nodes

Synchronization challenges

- # race conditions & consistency challenges

- #Distributed Lock

CAP Theorem

- # only have 2 of 3 properties: Consistency, Availability & Partition tolerance

- # Availability should be default choice

- # Strong consistency sys eg: inventory management, booking sys, banking sys

Locking

- # ensuring only 1 client can access a shared resource

- # Granularity of lock - lock only what is required and not whole object

- # Duration of lock - as short as possible

- # ByPass locks if work is read-only

- # Optimization concurrency.- assumptions that most of time we wont have contention

Index

Indexing

Making data faster to query

Tolerate slow writes but can't tolerate slow reads

makes reads faster

keep data in hash map by specific key

keep data in sorted list. Binary search to find data is $O(\log n)$

Specialized Indexes:

Geospatial index -> location data

Vector DB -> high dimensional data

Elastic Search -> full text search by

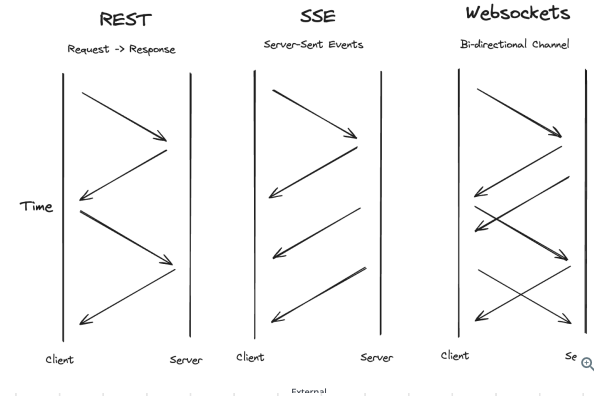
Communication Protocols

HTTPS- simple req/resp

Polling -> near realtime updates

Websockets -> realtime, bidirectional comm

SSE [Server sent event] -> unidirectional



Security

Authentication # Encryption # Data Protection # Monitoring-

> Infra, Service Level, Applicatoin level

Key Technologies

Core Database

1. Relational [Postgres] -> having relationships in data
2. NoSQL [DynamoDB]-> scale & performance

Relational DB

RDMS # Transactional #SQL Joins #Indexes #RDMS Transaction
Postgres #MySQL

NoSQL DB

Key-val, doc, col-family, graph forma
schema less # Flexible Data Models, Scalability , Big data, real time web apps
Things to know-> #Data Models # Consistency Models, # Indexing #Scalability
Eg-> DynamoDB, Cassandra, MongoDB

Blob Storage

store large, unstructured blobs of data . Image, video
Amazon S3, Google Cloud, Azure Blob
Scenario- Youtube, Instagram, Dropbox,

Search Optimized Database

use indexing, tokenization & stemming

Inverted Indexes:

data structure that maps from words to doc that contain them

Tokenization:

breaking piece of text into individual words

Stemming

process of reducing words to their root form. Running/Runs red. to run

Fuzzy Search:

find results that are similar to given search term

Eg- Elastic search, Restful search built on Apache

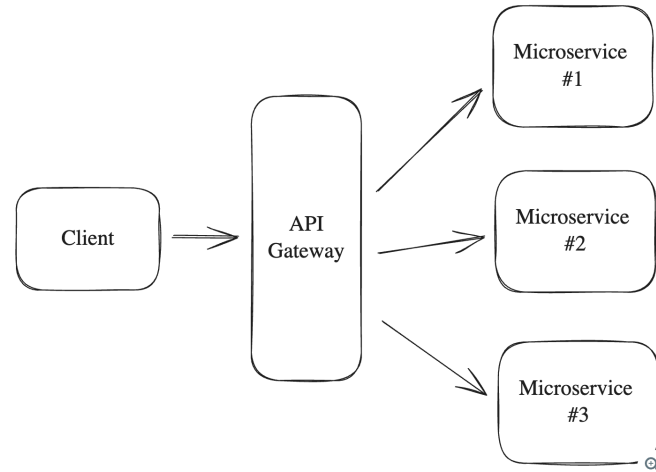
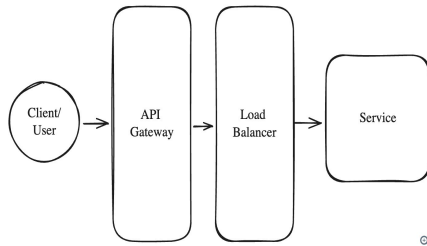
API Gateway:

common api - AWS , Kong, Apigess

Load Balancer:

Sticky sessions

Persistent conn



Common Load Balancer

AWS Elastic LB

NGINX

HAProxy

Streams:

Apache Flink, Spark

Stream

Kafka

Scaling with partition across multiple servers

multiple consumers groups

replication

windowing

Kafka

Queue

serve as buffers for bursty traffic

decouple producers & consumers of sys

Buffer for Bursty Traffic

Uber sudden surge- Q buffer -> process @ manageable rate w/o overloading server

Distribute work across sys

distribute expensive image processing task

Features of Q

Message Ordering - FIFO. **Kafka**

Retry Mechanism - retries certain no of times before considering a failure

Dead Letter Q- store messages that can't be processed. **Debugging & Auditing**

Scaling with partition- partition across servers.

Backpressure- way of slowing down production of message when Q is overwhelmed. If Q is full - reject or slow down rate new

Kafka , SQS

Kafka- distributed streaming platform

SQS- fully managed Q service by AWS

Distributed Locks

lock smt across sys for reasonale period

using distributed key value store like Redis or Zookeeper

set expiry.

Ecommerce, Ride sharing, Cron Job, Auction bidding

Mechanism

-> Redlock from Redis

->lock expiry -> locking grnularity -> deadlocks

Distributed Cache

Need-> save time, reduce no of DB query, speed up expensive Queries

Things to know:

#Eviction Policy: 1. LRU (Least Recently Used), 2. FIFO, 3.LFU (Least Frquently Used)

#Cache Invalidation Strategy: ensure data in cache is upto date.

#Cache Writing Strategy:

1. Write Through Cache: write to both cache & underlying db simult.

2. Write Around Cache: directly to datastore, bypassing cache.

Increase fetch time

3. Write- Back Cache: writes data to cache & then asyc to datastore

Cache Technologies: 1. Redis 2. Memcache
Redis: key-val store support string, hashes, list, sets, **Memcaches:** key valye store support strings

CDN:
A content delivery network (CDN) is a type of cache that uses distributed servers to deliver content to users based on their geographic location.
work by caching content on servers that are close to users
common application of a CDN in an interview is to cache static media assets like images and videos

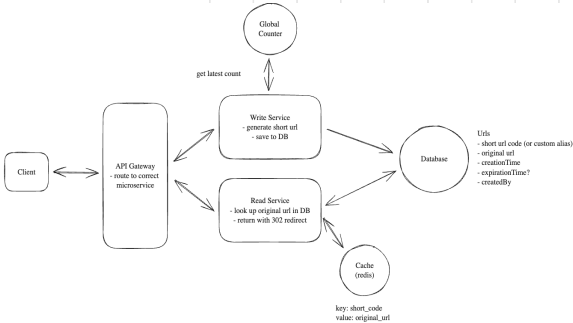
Eviction Policy ~~TTL~~-Time to Live
Eg: Cloudflare , Akamai, Amazon CloudFront

9/10 you'll use a REST API and focus on choos

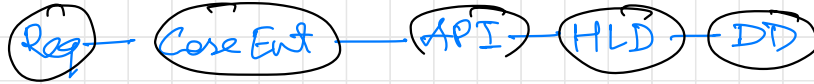
- **POST:** Create a new resource
- **GET:** Read an existing resource
- **PUT:** Update an existing resource
- **DELETE:** Delete an existing resource

Side-by-Side:

Feature	Layer 4	Layer 7
Works on	TCP/UDP	HTTP/HTTPS
Smart about request data?	❌ No	✅ Yes
Speed	⚡ Faster (low overhead)	🧠 Smarter (more logic)
Use case	Gaming, streaming	Web apps, APIs
Load decision	Based on IP/Port	Based on URL, headers, cookies



Drop Box



① Req

Funcⁿ

- upload file
- download file
- auto syn file across services.

② Core Entity

- File raw
- File metadata
- Users

③ API

Out of Scope

- roll own blob

Non-Functⁿ

- avail >> consistent (CAP)
- low latency upload & download
- support large file
- resumable upload

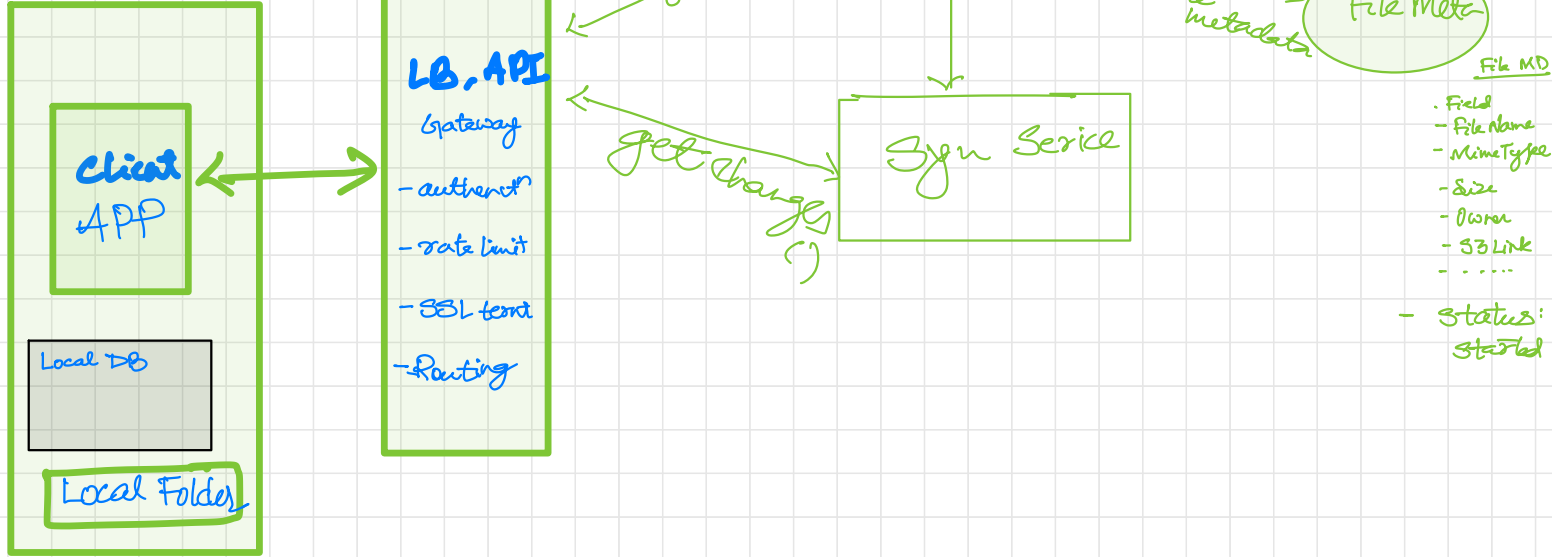
① Post file → 200
body: file & meta data [user in header]

Get Files / FileId →
File & FileMeta

Get /changes? since=qts? → Fields[]

- high data integrity (sync)

HDL



- Remote change

- Pull for changes
- download

- Local change

- upload changed cases

Windows API / Mac

- File Sync Watcher

upload large file

50 gb

Microblogging Twitter

